

Singleton Design Pattern - Complete Notes

[Singleton Design Pattern — Complete Notes](#)

- [1. First Principles — Why Singleton Exists](#)
- [2. Real-Life Analogies](#)
- [3. Core Mechanics — How It Actually Works](#)
- [4. Building It Up Step by Step \(Java Examples\)](#)
- [5. Types of Singleton \(By Initialization Strategy\)](#)
- [6. Real-World Usage Examples](#)
- [7. Counter-Questions Interviewers Ask \(with Detailed Answers\)](#)
- [8. FAANG-Style Interview Question — Full Walkthrough](#)

Singleton Design Pattern — Complete Notes

1. First Principles — Why Singleton Exists

Imagine building an application that needs a class representing configuration settings, a database connection manager, or a logging system. If multiple parts of the program each create their own instance of such a class, the app ends up with multiple disconnected copies of the same conceptual “thing.” One part changes a setting; another part never sees the change because it is looking at a different object in memory. This causes inconsistency, wasted memory, and bugs that are extremely hard to trace.

Core problem solved: Ensure exactly one instance of a class exists for the entire lifetime of the program, and every part of the code that asks for this object receives the same object — not a copy.

That single guarantee is the entire motivation. Every implementation detail below exists only to enforce this guarantee correctly and efficiently.

2. Real-Life Analogies

The Prime Minister’s Office. There is exactly one Prime Minister at any time. No matter which ministry, department, or citizen refers to “the Prime Minister,” they are all referring to the same person — not a freshly created clone each time. The “instance” is the actual person; the “global access point” is the title/office; the “controlled creation” is the strict election process that prevents arbitrary new PMs from appearing.

The Office Printer Spooler. If every application on every computer created its own printer driver instance and talked to the physical printer independently, print jobs would overlap and garble. Instead, one printer spooler object manages the queue, and everyone sends jobs to that single spooler.

The Database Connection Pool Manager. Creating a database connection is expensive — TCP handshake, authentication, memory allocation. If every class needing the database created its own connection, the database server’s connection limit would be exhausted almost immediately. Instead, one Singleton object manages and distributes connections from a shared pool.

3. Core Mechanics — How It Actually Works

Three elements must work together to guarantee “only one instance ever exists”:

1. **Private constructor.** Nobody outside the class can call `new MyClass()`. This is the controlled-creation part. A public constructor would let anyone create unlimited instances, defeating the purpose entirely.
 2. **Static instance variable.** The class itself holds a reference to its one and only instance. Because this variable belongs to the class (not to any object), it persists for the program’s lifetime and is shared across all code that references the class.
 3. **Static access method — typically `getInstance()`.** Since the constructor is private, this is the only door through which the outside world can obtain the object. It checks: does the instance already exist? If yes, return it. If no, create it once, store it, and then return it.
-

4. Building It Up Step by Step (Java Examples)

Step 1 — Eager Initialization (Naive)

```
public class ConfigurationManager {
    private static final ConfigurationManager instance = new ConfigurationManager();

    private Map<String, String> settings;

    private ConfigurationManager() {
        settings = new HashMap<>();
        settings.put("appName", "MyApp");
        settings.put("version", "1.0");
    }

    public static ConfigurationManager getInstance() {
        return instance;
    }

    public String getSetting(String key) {
        return settings.get(key);
    }
}
```

Why it works: The moment the JVM loads the class, `instance` is created exactly once. Being `static final` prevents reassignment, and the private constructor prevents external creation.

Why it can be bad: This is eager initialization — the object is created the instant the class loads, even if `getInstance()` is never called. If the constructor performs expensive work (loading large config files, opening network connections), that cost is paid upfront, possibly for nothing.

Step 2 — Lazy Initialization

```

public class ConfigurationManager {
    private static ConfigurationManager instance;

    private ConfigurationManager() {
        // expensive setup here
    }

    public static ConfigurationManager getInstance() {
        if (instance == null) {
            instance = new ConfigurationManager();
        }
        return instance;
    }
}

```

Why it's better: The object is created only the first time `getInstance()` is called, saving memory and startup time if it's never needed.

Why it's broken in real applications: Not thread-safe. If two threads call `getInstance()` at nearly the same moment, both may see `instance == null` and both proceed to create a new object — producing two instances and violating the entire Singleton guarantee. This is a classic race condition.

Step 3 — Thread-Safe Singleton (Synchronized Method)

```

public class ConfigurationManager {
    private static ConfigurationManager instance;

    private ConfigurationManager() {}

    public static synchronized ConfigurationManager getInstance() {
        if (instance == null) {
            instance = new ConfigurationManager();
        }
        return instance;
    }
}

```

Why it works: `synchronized` ensures only one thread executes `getInstance()` at a time, so simultaneous calls cannot both create instances.

Why it's still not ideal: Synchronization carries a performance cost on every call — even the millionth call long after the instance was created. Locking is only truly necessary during first creation, so this is wasted overhead in high-traffic applications.

Step 4 — Double-Checked Locking (Production-Grade)

```

public class ConfigurationManager {
    private static volatile ConfigurationManager instance;

    private ConfigurationManager() {}

    public static ConfigurationManager getInstance() {
        if (instance == null) { // first check (no lock, fast path)
            synchronized (ConfigurationManager.class) {
                if (instance == null) { // second check (inside lock)
                    instance = new ConfigurationManager();
                }
            }
        }
        return instance;
    }
}

```

Why two checks? - The **first check** (no lock) is the fast path. Once `instance` is non-null — true for the overwhelming majority of calls after startup — the method returns immediately without touching the lock. - The **second check** (inside the lock) handles the rare case where two threads both pass the first check while `instance` is still null. The lock ensures only one creates the object; the other, after acquiring the lock, sees the instance is no longer null and skips creation.

Why volatile? Object creation `instance = new ConfigurationManager()` is not atomic — it involves: (1) allocate memory, (2) initialize fields, (3) point `instance` to that memory. Without `volatile`, compiler/CPU reordering could let another thread see `instance` as non-null (step 3 done) before initialization (step 2) completes — that thread would then use a half-constructed object. `volatile` prevents this reordering and guarantees visibility across threads.

Step 5 — Bill Pugh Holder Pattern (Best Practice)

```

public class ConfigurationManager {
    private ConfigurationManager() {}

    private static class Holder {
        private static final ConfigurationManager INSTANCE = new ConfigurationManager();
    }

    public static ConfigurationManager getInstance() {
        return Holder.INSTANCE;
    }
}

```

Why it's elegant: The JVM guarantees a class is loaded — and its static fields initialized — only when first referenced. `Holder` is never referenced until `getInstance()` is first called. Result: - Lazy (instance not created until needed) - Thread-safe without explicit synchronization (class loading itself is thread-safe by JVM spec) - Fast (no locks, no volatile checks needed)

This achieves everything double-checked locking achieves, more cleanly. Most senior engineers consider this the “correct” Java approach.

Step 5b — Enum Singleton

```

public enum ConfigurationManager {
    INSTANCE;

    private Map<String, String> settings = new HashMap<>();

    public String getSetting(String key) {
        return settings.get(key);
    }
}

// usage: ConfigurationManager.INSTANCE.getSetting("appName");

```

Why it works: Java guarantees each enum constant is instantiated exactly once, and the JVM automatically protects enums against serialization and reflection attacks. Joshua Bloch (*Effective Java*) recommends this as the best Java approach when applicable.

5. Types of Singleton (By Initialization Strategy)

Type	Description	When to use
Eager Initialization	Instance created at class load time	Lightweight object, always needed
Lazy Initialization	Instance created on first use	Expensive object, might not always be needed
Thread-Safe (synchronized)	Lazy init + locking on every call	Multi-threaded apps where correctness > raw speed
Double-Checked Locking	Lazy init + locking only during creation	High-frequency <code>getInstance()</code> calls in multi-threaded apps
Bill Pugh / Holder	Lazy + thread-safe + no locking overhead	Generally the gold standard in Java
Enum Singleton	Simplest, safest against reflection/serialization	When extensibility (inheritance) is not needed

6. Real-World Usage Examples

- **Logger objects** (e.g., `java.util.logging.Logger`, Log4j) — all parts of an app write to the same log stream with consistent configuration.
- **Database Connection Pool** (e.g., HikariCP) — manages a fixed, shared set of expensive connections.
- **Cache managers** — a single shared in-memory store, not duplicated per class.
- **Thread pools** (`Executors`) — coordinate a limited, shared set of worker threads.
- **Application configuration/settings objects** — as in the analogy above.
- **Spring Framework beans** — Spring beans are Singleton-scoped within the application context by default (Spring manages this for you).
- **Hardware interface drivers** — a single object representing access to a printer or GPU, since the underlying hardware resource is itself singular.

7. Counter-Questions Interviewers Ask (with Detailed Answers)

Q1. Isn't Singleton just a glorified global variable?

A global variable is created at program start with no control over creation timing or access. A Singleton adds: lazy initialization control; encapsulation (internal state hidden behind methods); the ability to implement interfaces or extend classes (enabling polymorphism); and controlled, single-

point creation logic (e.g., reading config, validation). That said, Singleton shares many criticisms of globals — global mutable state, hidden dependencies, hard to test.

Q2. Why is Singleton often considered an anti-pattern?

- **Hidden dependencies:** A class calling `Logger.getInstance()` internally doesn't declare this dependency in its constructor, making it unclear from the outside what the class relies on.
- **Testing difficulty:** Unit tests need to substitute mocks for dependencies. A class hardcoding `Database.getInstance()` can't easily receive a mock database during testing — violating dependency injection principles.
- **Global mutable state:** If the Singleton holds mutable data, any code can change it, and tracking who changed it and when becomes very difficult — especially under concurrency.
- **Tight coupling:** Code becomes coupled to a concrete class rather than an abstraction, violating the Dependency Inversion Principle.
- **Violates Single Responsibility Principle:** The class manages both its core business logic and its own lifecycle/instantiation — two responsibilities.

Modern alternative: use a Dependency Injection framework (Spring, Guice, Dagger) to manage a single instance and inject it where needed — gaining the “one instance” benefit without hardcoded global access or testability problems.

Q3. How can a Singleton be broken — ending up with two instances?

- **Reflection:** `Constructor.setAccessible(true)` can call a private constructor directly, bypassing `getInstance()`. Defense: throw an exception in the constructor if `instance != null` already.
- **Serialization/Deserialization:** If the Singleton implements `Serializable`, `ObjectInputStream.readObject()` creates a brand-new object, not the original. Defense: implement `readResolve()` to return the existing instance.
- **Cloning:** If the class implements `Cloneable`, `.clone()` creates a second instance. Defense: override `clone()` to throw `CloneNotSupportedException`.
- **Multiple classloaders:** The same class loaded by two different classloaders is treated as two different classes by the JVM, each getting its own static instance — resulting in two Singletons.
- **Enum Singleton** avoids the reflection and serialization problems via JVM built-in protections specific to enums.

Q4. How would you test a class that depends on a Singleton?

This is genuinely hard — part of the criticism. Approaches: add a package-private setter to inject a mock instance for tests (somewhat hacky); extract an interface the Singleton implements so the dependent class depends on the interface, allowing a mock implementation to be passed; or use a DI framework where “singleton scope” is container-managed, making substitution in tests trivial.

Q5. Does Singleton still make sense in microservices/distributed systems?

Within a single process/JVM, yes — it guarantees one instance per process. But across multiple servers running instances of your service (normal in distributed systems), each process has its own Singleton — “one instance” is per-process, not global across the fleet. For a logical entity that must be unique across an entire distributed system (e.g., distributed lock, leader node), use a distributed coordination service (ZooKeeper, etcd) implementing leader election — a fundamentally different problem from the in-process Singleton.

Q6. Singleton vs. a class with only static methods — why not just use static methods everywhere?

A static-only class cannot implement interfaces, cannot be passed around as an object reference, cannot leverage inheritance/polymorphism for its state, and cannot be lazily initialized with complex constructor setup logic. A Singleton is a real object — it can implement interfaces (enabling abstraction and easier testing), maintain instance state cleanly, and participate in object-oriented designs such as being passed via dependency injection.

8. FAANG-Style Interview Question — Full Walkthrough

Prompt: “Design a thread-safe Singleton class for a Logger that writes messages to a file. Discuss tradeoffs of at least two implementations, and explain how you’d prevent it from being broken via reflection.”

Expected discussion points:

1. Start with the basic structure: private constructor, static instance, static `getInstance()` .
2. Discuss lazy vs. eager — a Logger is lightweight, so eager init may be acceptable, but if it opens a file handle on construction, lazy is safer.
3. Propose double-checked locking OR the Bill Pugh holder pattern, explaining why (visibility guarantees, performance, as detailed in Step 4 and Step 5).
4. If using double-checked locking, mention `volatile` and explain the instruction-reordering risk it prevents.
5. Address the reflection attack: add a check in the constructor — `if (instance != null) throw new IllegalStateException("Instance already created");`
6. Bonus depth: discuss flushing/closing the file on JVM shutdown via `Runtime.getRuntime().addShutdownHook(...)` .
7. Bonus: discuss whether an Enum Singleton fits here — generally yes, since a logger rarely needs to extend another class, and enums offer strong built-in protection.

A strong candidate connects each implementation choice back to a concrete consequence — performance, correctness, or security — rather than just reciting syntax. That distinction separates memorized answers from genuine understanding.